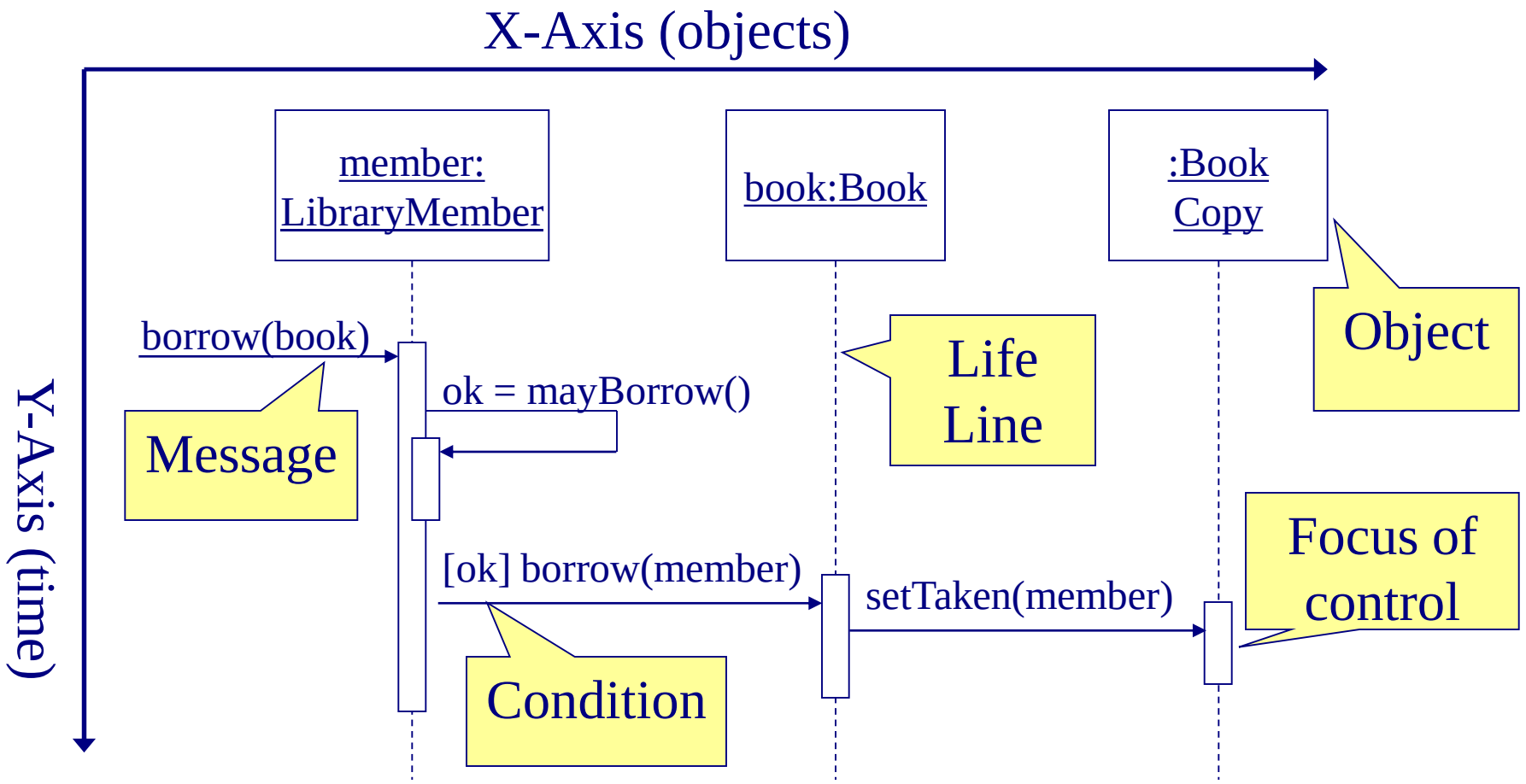
# Sequence Diagram

# Sequence Diagram

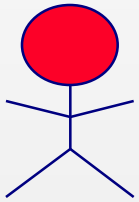
Illustrates the objects that participate in a use case and the messages that pass between them over time for *one* use case

In design, used to distribute use case behavior to classes

# Sequence Diagram



# Sequence Diagram Syntax

| | |
|---|---|
| AN ACTOR |  |
| AN OBJECT | **anObject:aClass** |
| A LIFELINE |  |
| A FOCUS OF CONTROL |  |
| A MESSAGE | aMessage() |
| OBJECT DESTRUCTION | x |

# Sequence Diagram

Two major components

- Active objects
- Communications between these active objects
  - Messages sent between the active objects

# Sequence Diagram

Active objects

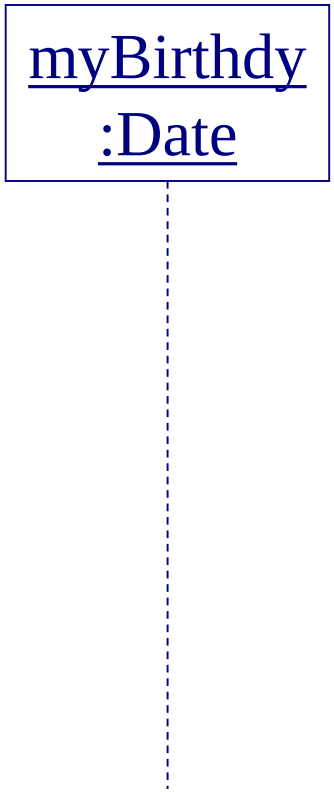
- Any objects that play a role in the system
- Participate by sending and/or receiving messages
- Placed across the top of the diagram
- Can be:
  - An actor (from the use case diagram)
  - Object/class (from the class diagram) within the system

# Active Objects

## Object

- Can be any object or class that is valid within the system
- Object naming
  - Syntax

**[instanceName][:className]**



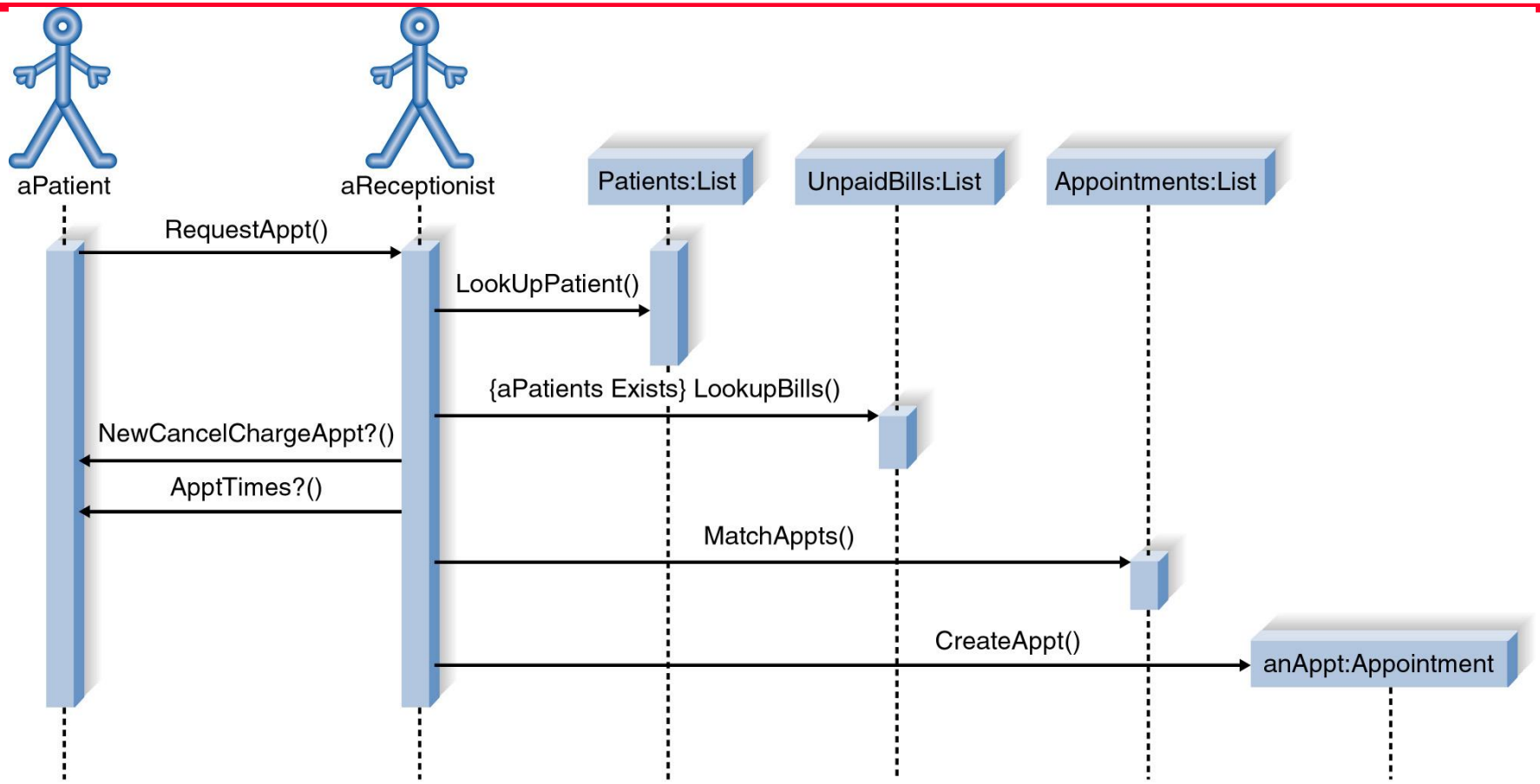

1. Class name only :Classname
2. Instance name only objectName
3. Instance name and class name together object:Class

# Active Objects

## Actor

- A person or system that derives benefit from and is external to the system
- Participates in a sequence by sending and/or receiving messages

# Sequence Diagram



# Communications between Active Objects

## Messages

- Used to illustrate communication between different active objects of a sequence diagram
- Used when an object needs
  - to activate a process of a different object
  - to give information to another object

# Messages

A message is represented by an arrow between the life lines of two objects.

- Self calls are allowed

A message is labeled at minimum with the message name.

- Arguments and control information (conditions, iteration) may be included.

# Types of Messages

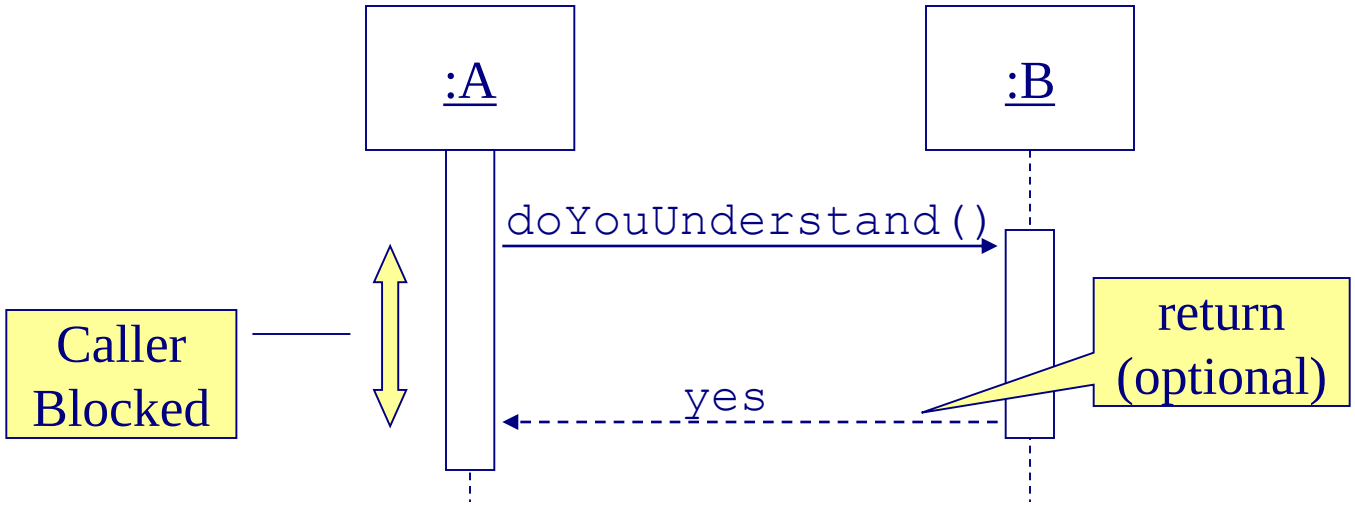
- Synchronous (flow interrupt until the message has completed)

- Asynchronous (don't wait for response)

- Return (control flow has returned to the caller)

# Synchronous Messages

The routine that handles the message is completed before the calling routine resumes execution.



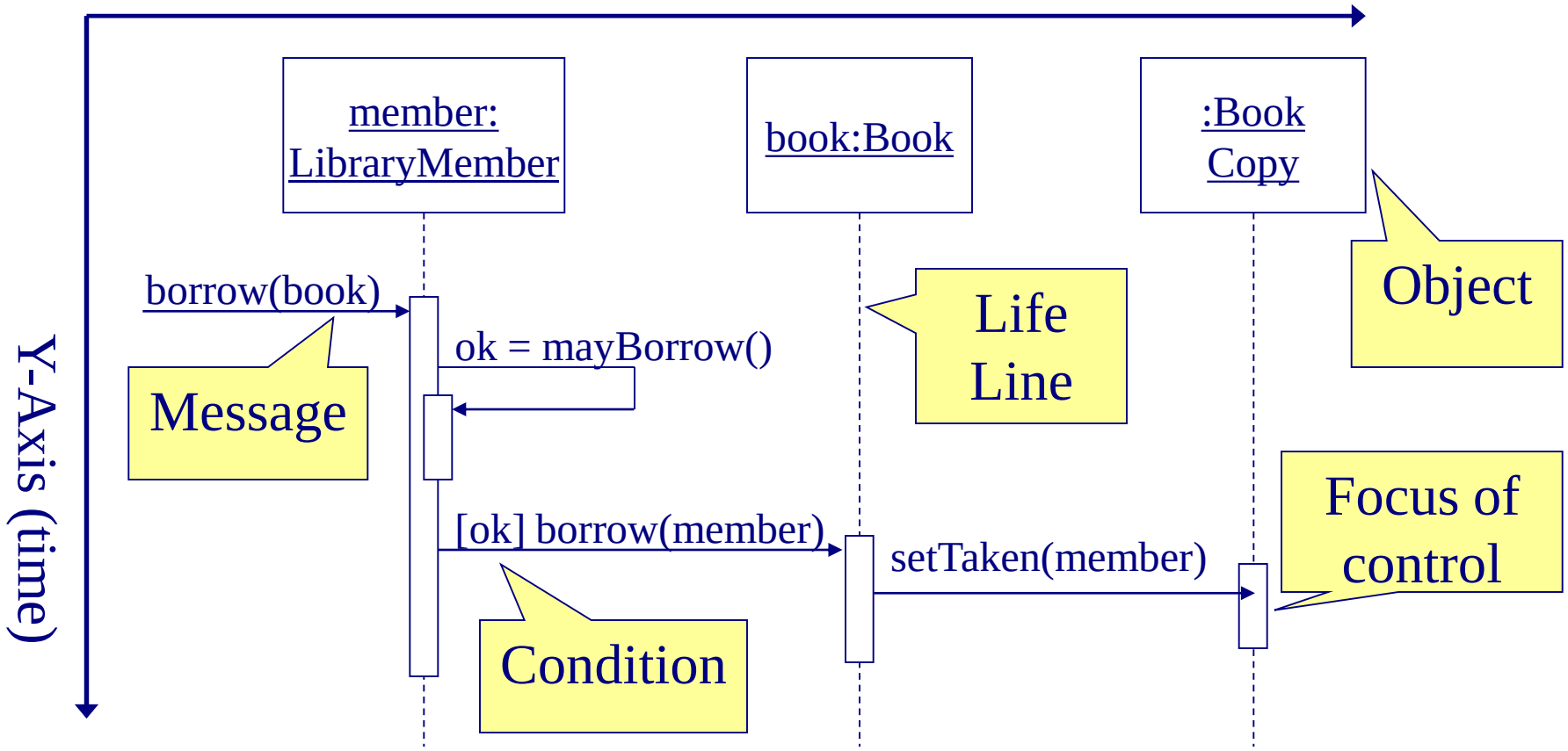
# Asynchronous Messages

- Calling routine does not wait for the message to be handled before it continues to execute.
  - As if the call returns immediately
- Examples
  - Notification of somebody or something
  - Messages that post progress information

# Return Values

- Optionally indicated using a dashed arrow with a label indicating the return value.
  - Don't model a return value when it is obvious what is being returned, e.g. getTotal()
  - Model a return value only when you need to refer to it elsewhere (e.g. as a parameter passed in another message)
  -

# Sequence Diagram



# Other Elements of Sequence Diagram

- Lifeline
- Focus of control (activation box or execution occurrence)
- Control information
  - Condition, repetition

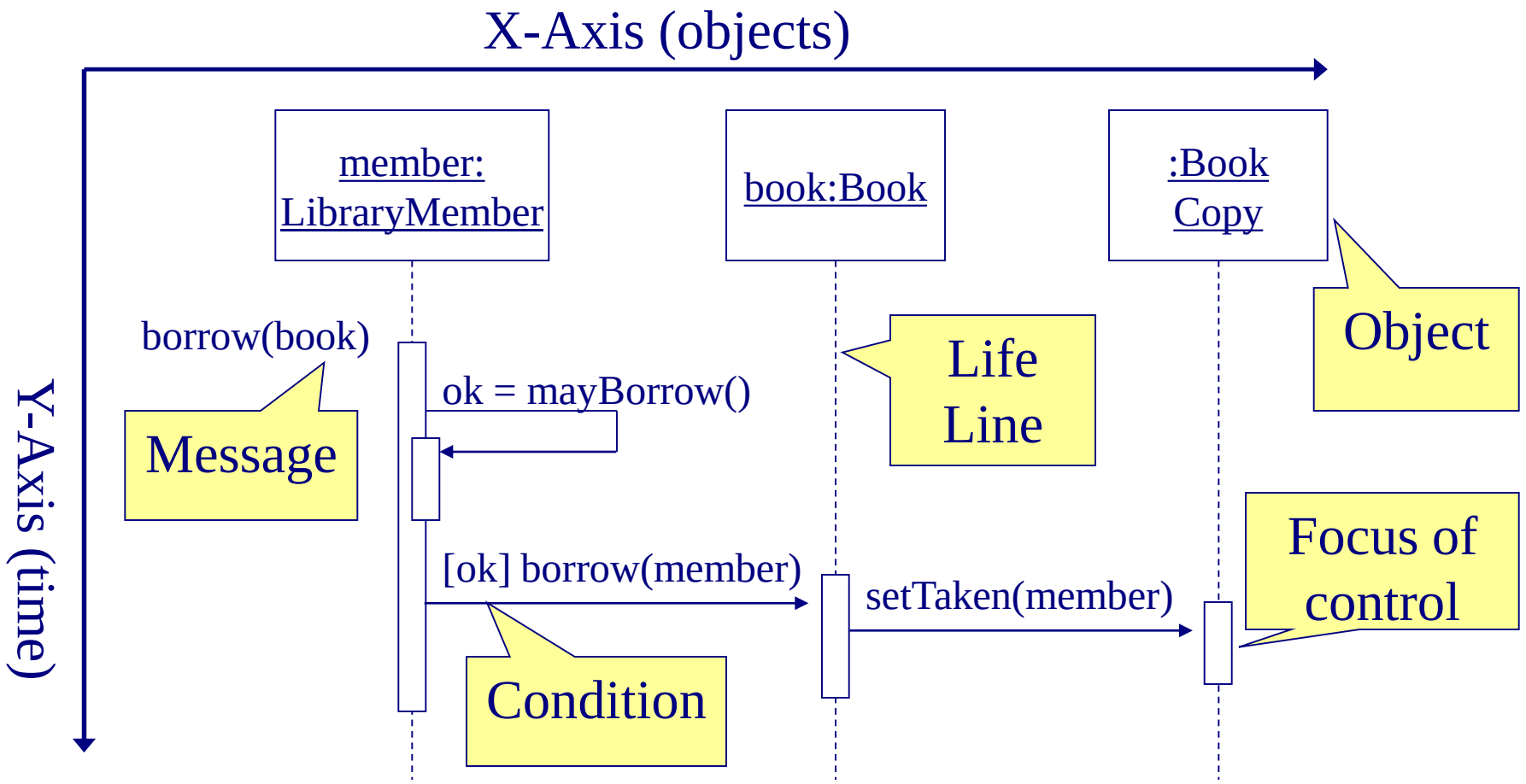
# Sequence Diagram

- Lifeline
  - Denotes the life of actors/objects over time during a sequence
  - Represented by a vertical line below each actor and object (normally dashed line)
- For temporary object
  - place X at the end of the lifeline at the point where the object is destroyed

# Sequence Diagram

- Focus of control (activation box)
  - Means the object is active and using resources during that time period
  - Denotes when an object is sending or receiving messages
  - Represented by a thin, long rectangular box overlaid onto a lifeline

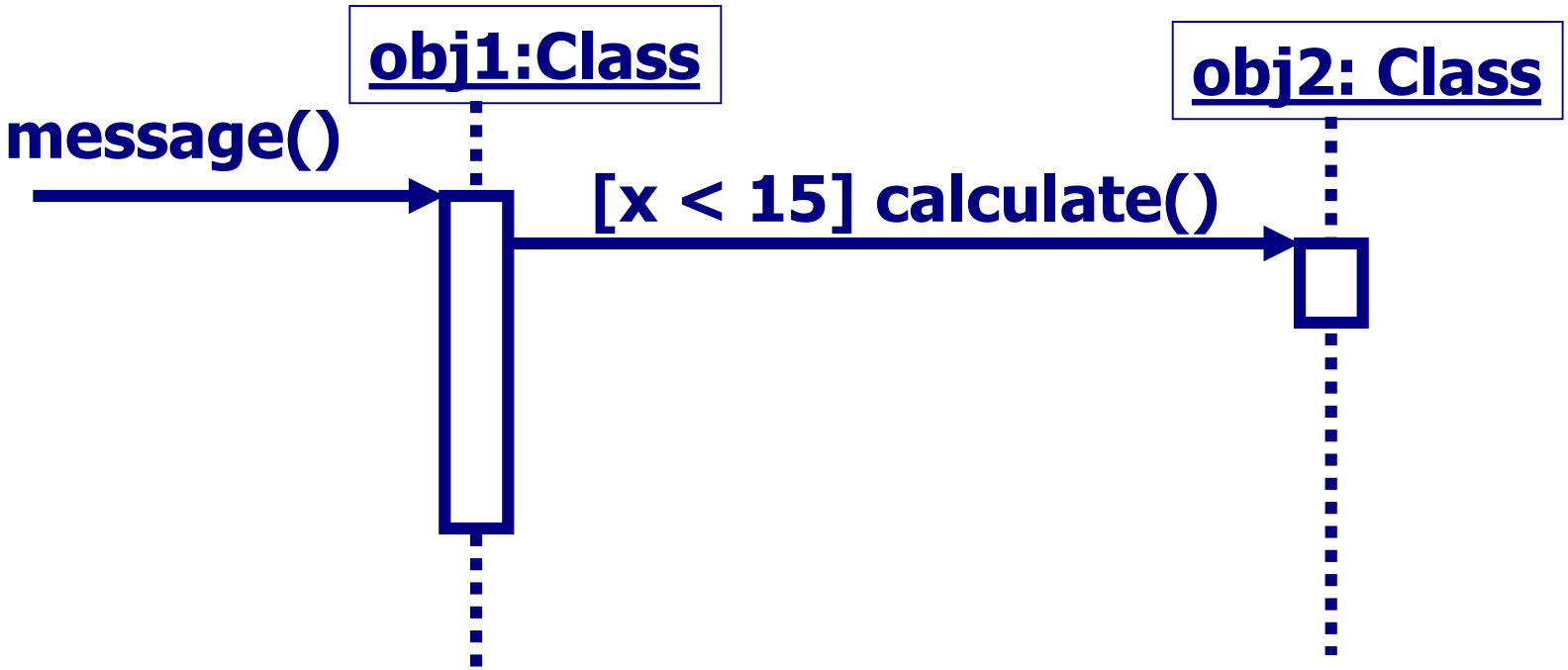
# Sequence Diagram



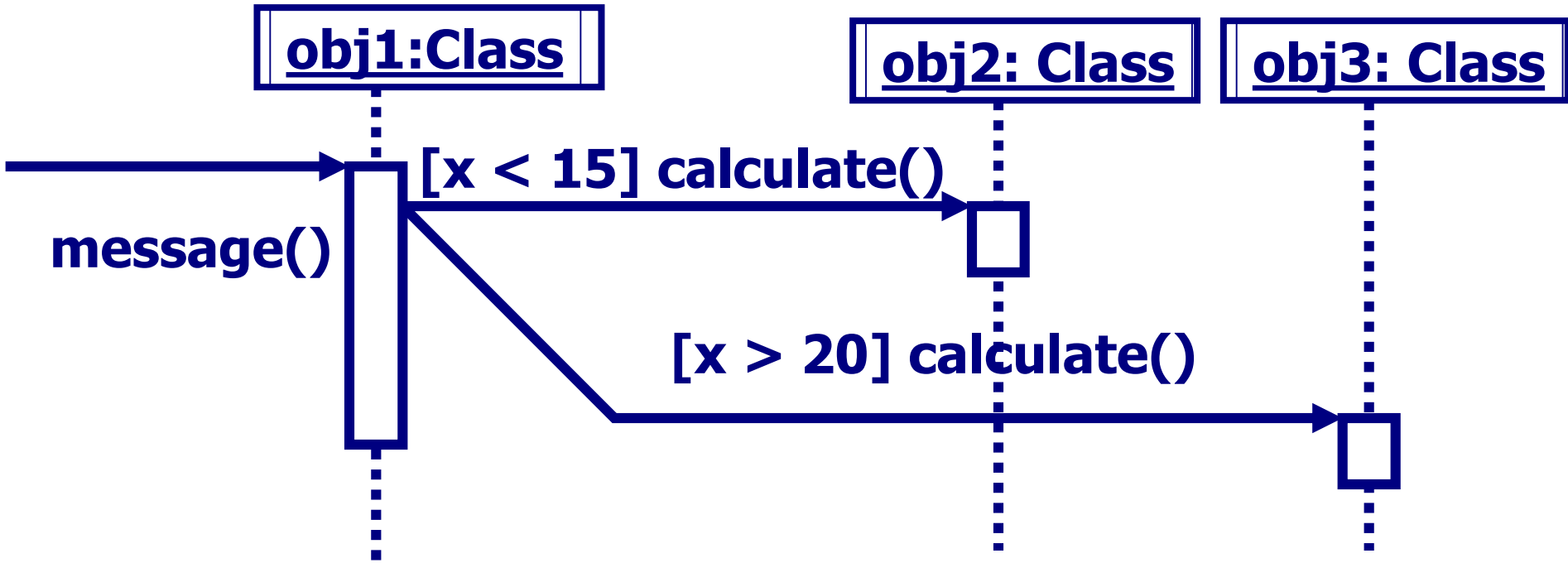
# Control Information

- Condition
  - syntax: ‘[‘ expression ’]’ message-label
  - The message is sent only if the condition is true

# Elements of Sequence Diagram

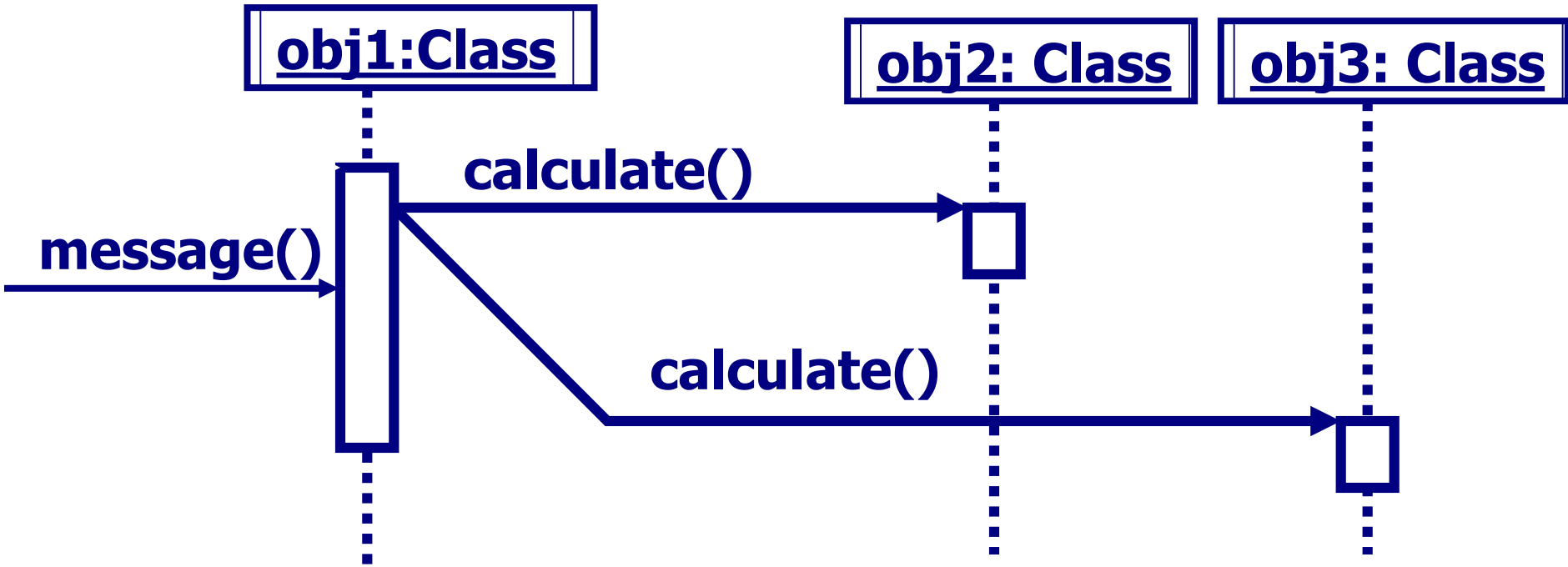


# Sequence Diagrams



# Sequence Diagrams

## Concurrency



# Elements of Sequence Diagram

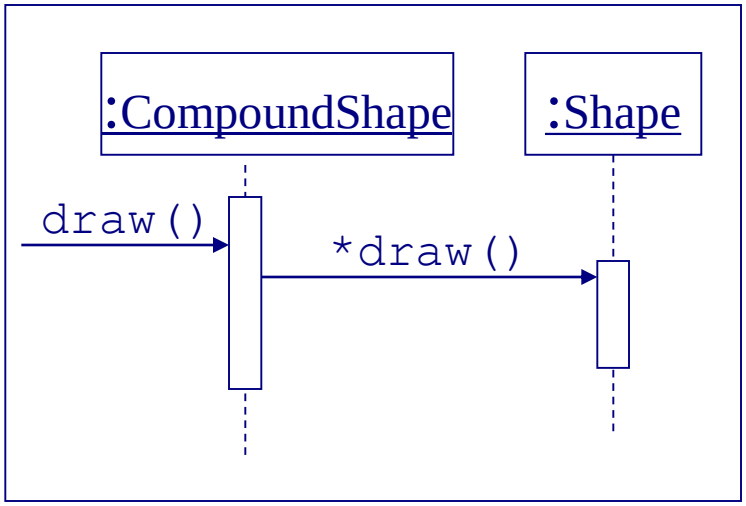
- Control information
  - Iteration
    - may have square brackets containing a continuation condition (until) specifying the condition that must be satisfied in order to exit the iteration and continue with the sequence
    - may have an asterisk followed by square brackets containing an iteration (while or for) expression specifying the number of iterations

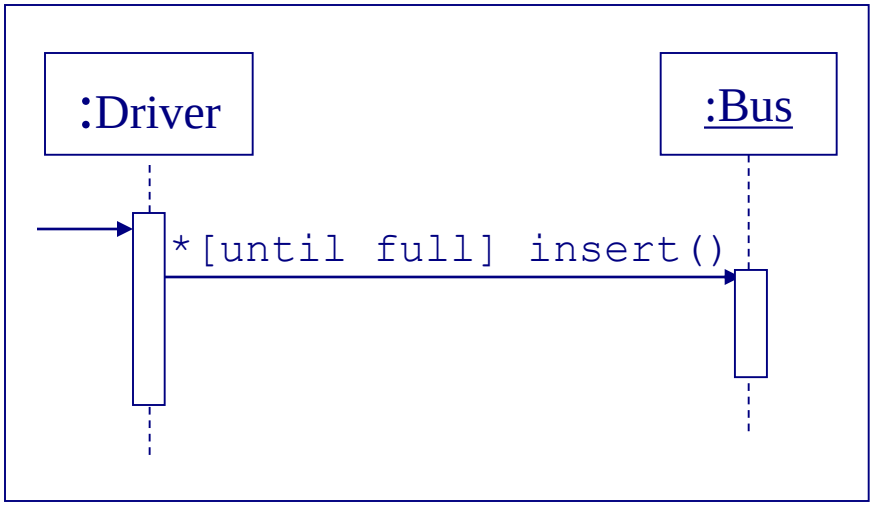
# Control Information

- Iteration
  - syntax: * [ '[' expression ']' ] message-label
  - The message is sent many times to possibly multiple receiver objects.

# Control Information

- Iteration example



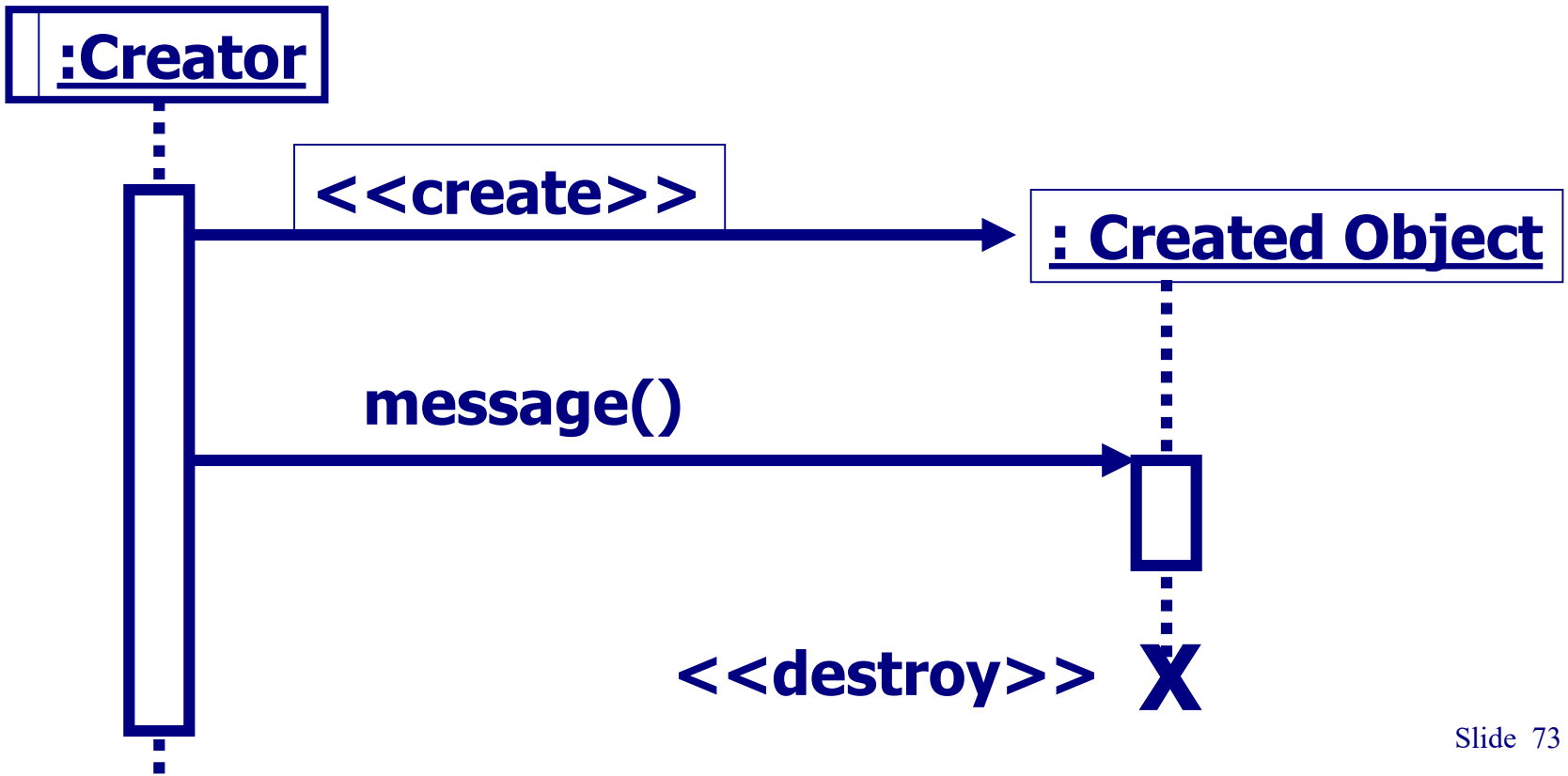



# Control Information

- The control mechanisms of sequence diagrams suffice only for modeling simple alternatives.
  - Consider drawing several diagrams for modeling complex scenarios.
  - Don’t use sequence diagrams for detailed modeling of algorithms (this is better done using *activity diagrams, pseudo-code* or *state-charts*).

# Sequence Diagrams

- Creation and destruction of an object in sequence diagrams are denoted by the stereotypes <<create>> and <<destroy>>

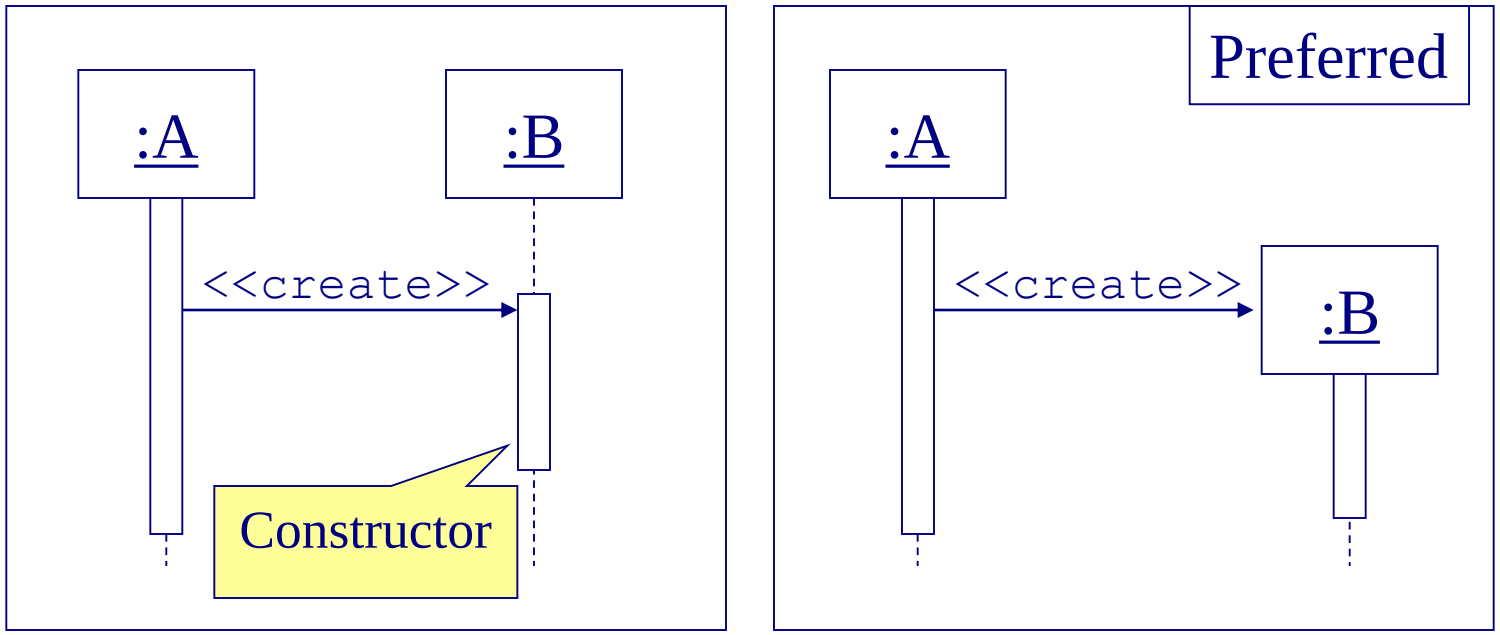


# Creating Objects

- Notation for creating an object on-the-fly
  - Send the <<create>> message to the body of the object instance
  - Once the object is created, it is given a lifeline.
    - Now you can send and receive messages with this object as you can any other object in the sequence diagram.

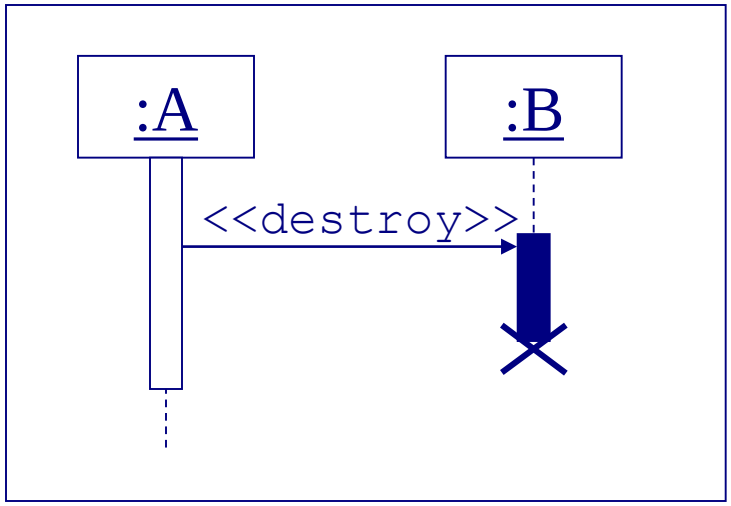
# Object Creation

- An object may create another object via a `<<create>>` message.

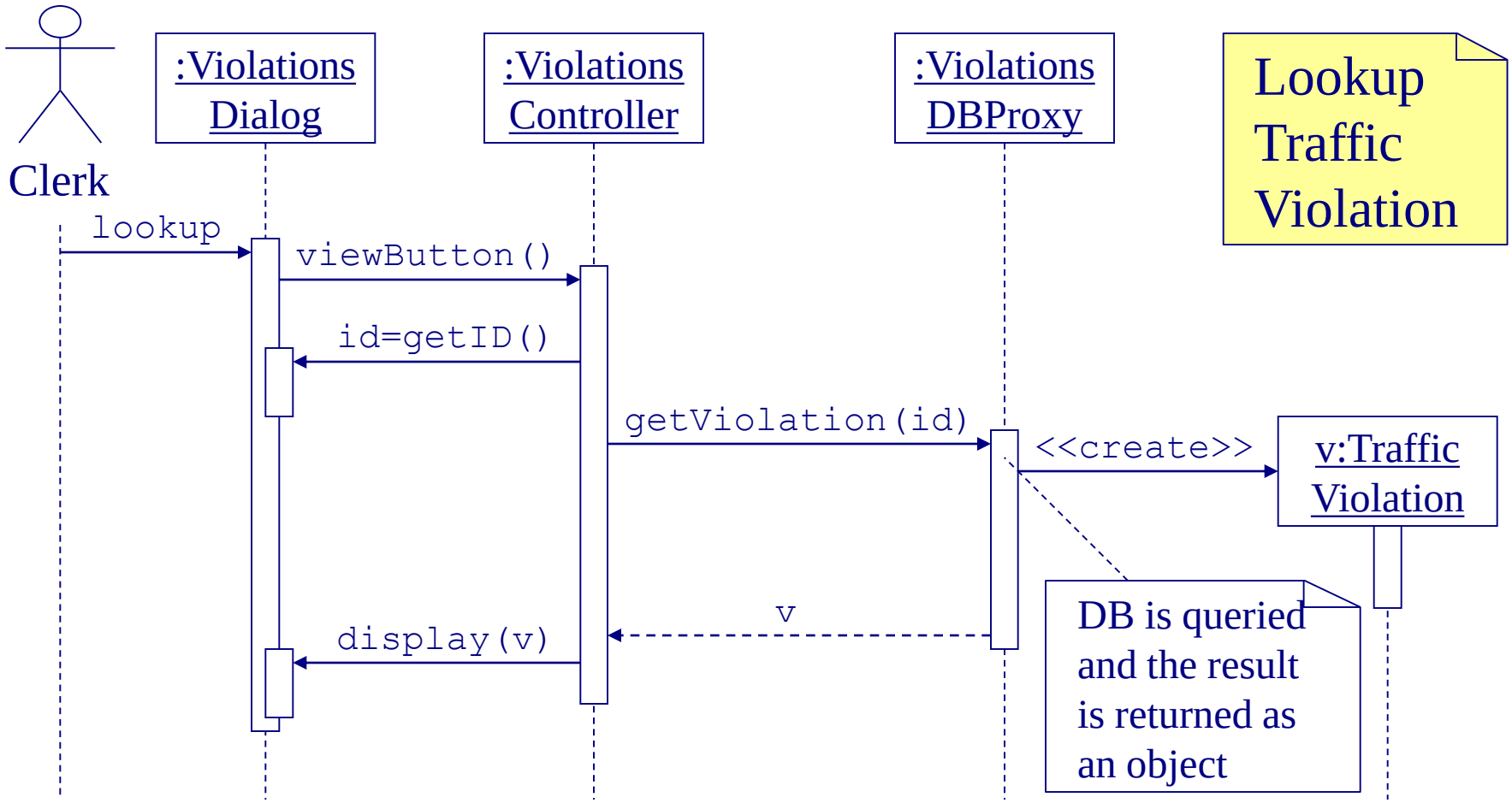


# Object Destruction

- An object may destroy another object via a `<<destroy>>` message.
  - An object may destroy itself.
  - Avoid modeling object destruction unless memory management is critical.



# Sequence Diagram



# Steps for Building a Sequence Diagram

1) Set the context
2) Identify which objects and actors will participate
3) Set the lifeline for each object/actor
4) Lay out the messages from the top to the bottom of the diagram based on the order in which they are sent
5) Add the focus of control for each object's or actor's lifeline
6) Validate the sequence diagram

# Steps for Building a Sequence Diagram

1) Set the context.
   a) Select a use case.
   b) Decide the initiating actor.

# Steps for Building a Sequence Diagram

2) Identify the objects that may participate in the implementation of this use case by completing the supplied message table.
   a) List candidate objects.
      1) Use case controller class
      2) Domain classes
      3) Database table classes
      4) Display screens or reports

# Steps for Building a Sequence Diagram

2) Identify the objects (cont.)

b) List candidate messages. (in message analysis table)

**1) Examine each step in the normal scenario of the use case description to determine the messages needed to implement that step.**

**2) For each step:**

1) Identify step number.
2) Determine messages needed to complete this step.
3) For each message, decide which class holds the data for this action or performs this action

**3) Make sure that the messages within the table are in the same order as the normal scenario**

# Steps for Building a Sequence Diagram

2) Identify the objects (cont.)
   c) Begin sequence diagram construction.
      1) Draw and label each of the identified actors and objects across the top of the sequence diagram.
      2) The typical order from left to right across the top is the actor, primary display screen class, primary use case controller class, domain classes (in order of access), and other display screen classes (in order of access)
3) Set the lifeline for each object/actor

# Steps for Building a Sequence Diagram

4) Lay out the messages from the top to the bottom of the diagram based on the order in which they are sent.

   Working in sequential order of the message table, make a message arrow with the message name pointing to the owner class.

   Decide which object or actor initiates the message and complete the arrow to its lifeline.

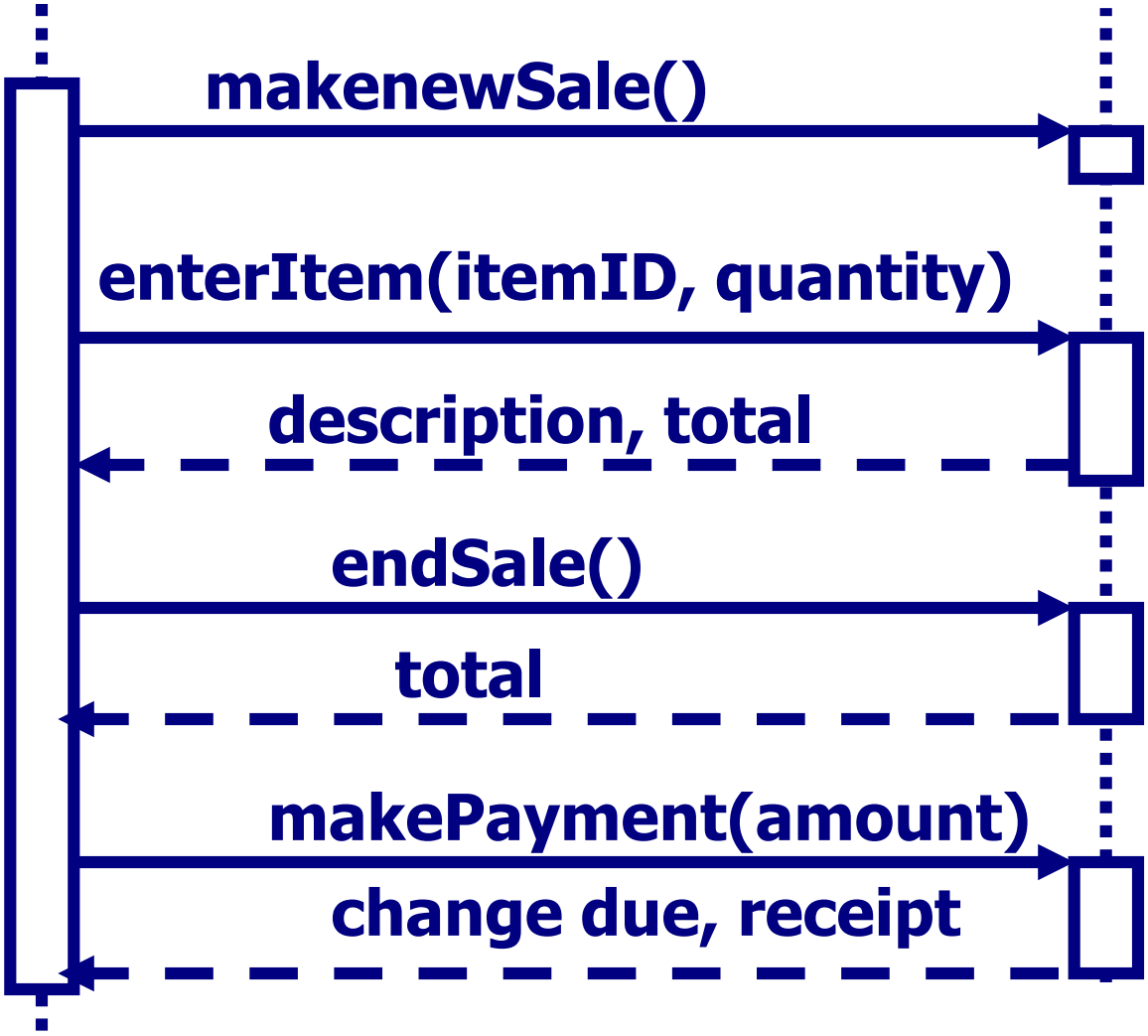
   Add needed return messages.

   Add needed parameters and control information.

# Steps for Building a Sequence Diagram

5) Add the focus of control (activation box) for each object's or actor's lifeline.
6) Validate the sequence diagram.

# Sequence Diagrams



# Sequence Diagram